Changing classes in saemix

Emmanuelle Comets

2025-07-10

Objective

Highlight changes in class structures and in entering the input between current version of saemix (3.x) and next version (4.0)

- several new classes created as building blocks to model classes
  - added a log slot to most classes to track warning messages so the user can review what changes were made during the creation of the object
  - class SaemixOutcome
    - define outcomes in the model (=responses)
    - used to compute gpred
  - class SaemixParameter
    - define parameters of the model
  - class SaemixVarModel
    - define variability model
  - class SaemixPopModel
    - define population-level individual parameters at each level of variability, eg for IIV $\phi_{pop,i}$
    - linear model ($(\phi_{pop,i} = \mu + \sum \beta.cov_i)$
  - class SaemixIndivModel
    - define individual parameters ϕ_i by adding random effects to $\phi_{pop,i}$)
    - includes the list of SaemixPopModel and SaemixVarModel objects
- SaemixData object
  - main changes
    - added a slot to specify the type of the outcomes
    - added a slot for grouping levels (varlevel)
    - added an associated covariate.varlevel to specify where the covariate model should enter
    - added a log slot to track warning messages so the user can review what changes were made during the creation of the object
  - additional changes
    - log all warning messages to print out or keep track of in the object
- **ToDo SaemixData Eco**
  - update legacy tests from testthat_saemixData-read.R
  - update legacy tests from testthat_saemixData-covariates.R
- **ToDo SaemixData Lillois**
  - check recording of error messages to make sure as much as possible is captured/tracked
  - update legacy creator so it still works with the new class (error messages in legacy tests)
  - add default behaviour to define name.group as varlevel[1] if given (potentially deprecate both name.group and name.occ)
  - program a function to extract the covariate matrices for the different levels in varlevel
    - input: the object
    - output: the object, with a slot holding the list of matrices with the first value of the covariate at each level

            - eg: for id, first value of the covariate in each subject (size: N x nb of covariates not varying across id)
            - for occ, first value of the covariate in each subject at each occasion (size Sum_i nocc_i x nb of covariates varying with occasion)
            - etc... if more levels of variability
            - add (and record) warning messages if the covariate changes value within a grouping level
- SaemixIndivModel (**new**)
  - split in two parts, using lists of the two new classes defining:
    - the individual population model ($\phi_{pop,i} = \mu + \sum \beta.cov_i$)
    - the variability model: structure of covariance-matrix Ω
  - functions associated with the variability model
  - functions associated with the individual population model
    - see notes and testthat
- SaemixModel object (**complete overhaul**)
  - child class of SaemixIndivModel
    - inherits the list of SaemixPopModel and SaemixVarModel objects
    - adds list of outcome in the model
    - adds the model function (and the sim.model function for non-Gaussian models)
  - **main changes**
  - multiple responses through the ytype identifier
    - definition of outcome through their distribution
      - continuous: normal
      - non-Gaussian: different types
    - error models for continuous responses
      - built-in error models
      - user-defined error models
  - definition of parameters through their distribution à la MLX-TRAN
    - built-in distributions with automatically defined transformation h and inverse h-1
    - in theory (untested), possible to write user-defined distributions
    - from the list of parameters, derive the variability levels
- **ToDo SaemixModel Lillois**
  - function to match varlevel between a model and an object
    - input: SaemixData and SaemixModel objects
    - output: TBD (could be both objects updated, or could be an updated varlevel and covariate.varlevel list for two functions to make the changes in the SaemixData and SaemixModel objects)
  - reprogram functions associated to the model class
    - predict: priority high, easy once matching between data and model done
    - summary: priority moderate, very easy
    - checkInitialFixedEffects: priority high, moderate difficulty once matching between data and model done
- **ToDo SaemixModel Eco**
  - reprogram plot functions associated to the model class
    - difficulty: high (dispatch according to response type)
    - code architecture involved
- input functions (**saemixData()** and **saemixModel()**)
  - legacy options to enter the model as previously
  - some new features
  - **ToDo someone**
    - create legacy function saemixModel.legacy()
    - test that previous models (no iov, single responses) give the same object (content) with the legacy and the new creator functions
- Fisher Information Matrix by linearisation
  - scope: **TBD**

  - **ToDo Lillois**
    1. equations for the FIM using h and gradient(h) (*thèse Sylvie Retout*)
       * function derivative of h added to the SaemixPar object, check if derivative of inverse transform function also needed
       * ToDo: write equations and implement them
    2. equations for the FIM in the presence of IOV (*thèse Caroline Bazzoli TBC, ou Anne Dubois, Florence , Kristofferson et al. 2015*)
       * biblio methods
       * write equations
       * implementation
    3. question: is the expression generalisable to more variability levels ??
  - additionally
    * FIM for multiple responses but already implemented by Lucas (?)
- further developments needed (out of scope for Lillois)
  - Eco
    * decide: structure for results or use the xxxHat children classes
    * add log slot in SaemixObject objects
    * make sure all warning messages are captured in the log slots for both data and model objects
  - Fisher Information Matrix by stochastic approximation (*Delattre et Kuhn Computo 2023*)
    * will need the same transformation as for the linearisation approach
    * extended to multiple responses by Alexandra
    * who: most likely Lucas or could be an intern project
  - testing user-defined error models
  - user-defined distributions
  - additional variability levels

Setup

- folders
  - set **workDir** to the working directory
  - needs the saemix root folder (here **/home/eco/work/saemix/saemixextension**)
  - data will be saved in folder **data40**
- libraries
  - need to install ggplot2 and its sister libraries
  - need packages testthat (tests)
  - saemix package functions are loaded, from two versions
    * current version: folder **R**
    * next version: folder **Rext**

Simulation of IOV with theophylline PK

- Features
  - theophylline PK
  - covariates
    * sex, weight changing with subject
    * CRCL, comedication changing with occasion
  - design
    * N=100 subjects
    * 1 to 3 occasion per subject (random)
    * sampling times for each occasion: 0.25, 1.5, 3.35, 12 and 24 h (sparse design in Dubois et al.)
    * dose: D=4mg
    * time between two occasions: 4 weeks (assume washout so no drug left)
- Model

  - PK response: 1 cpt model with absorption

$$Cpred = \frac{D}{V}\frac{k_a}{(k_a - CL/V)}\left(e^{-CL/Vt} - e^{-k_a t}\right)$$

  - log-normal parameter distribution
  - population parameter values (Dubois et al.)
    * ka=1.48 h-1
    * CL=0.04 L/h
    * V=0.48 L
  - variabilities
    * IIV
      · 30% on all parameters
      · correlation between CL and V (0.7)
    * IOV
      · IOV on CL only with 20% IOV
    * different from Dubois, where 2 scenarios, no correlation and no covariate effect
      · low IIV: 0.2 on ka and CL, 0.1 on V; half of IIV for IOV
      · regular BSV: 0.5 on ka, V, CL and 0.15 on IOV
    * combined residual error model with a=0.1 and b=0.1 (low IIV scenario from Dubois)
  - covariate models
    * weight: allometry on V and CL
    * gender: V decreased by 30% in women
    * CRCL: correlation with CL ($\beta_{CRCL,CL} = 0.5$)
    * comedication: increase in CL by 30%
- Not included in the simulation
  - NA values in the covariates
  - LOQ values

Read datasets

```
# Dataset without covariates
theonocov <- read.csv(file=file.path(datDir,"theoSimul_nocov.csv"), header=T, sep=",")

# Dataset with covariates
theocov <- read.csv(file=file.path(datDir,"theoSimul_cov.csv"), header=T, sep=",")
```

Saemix

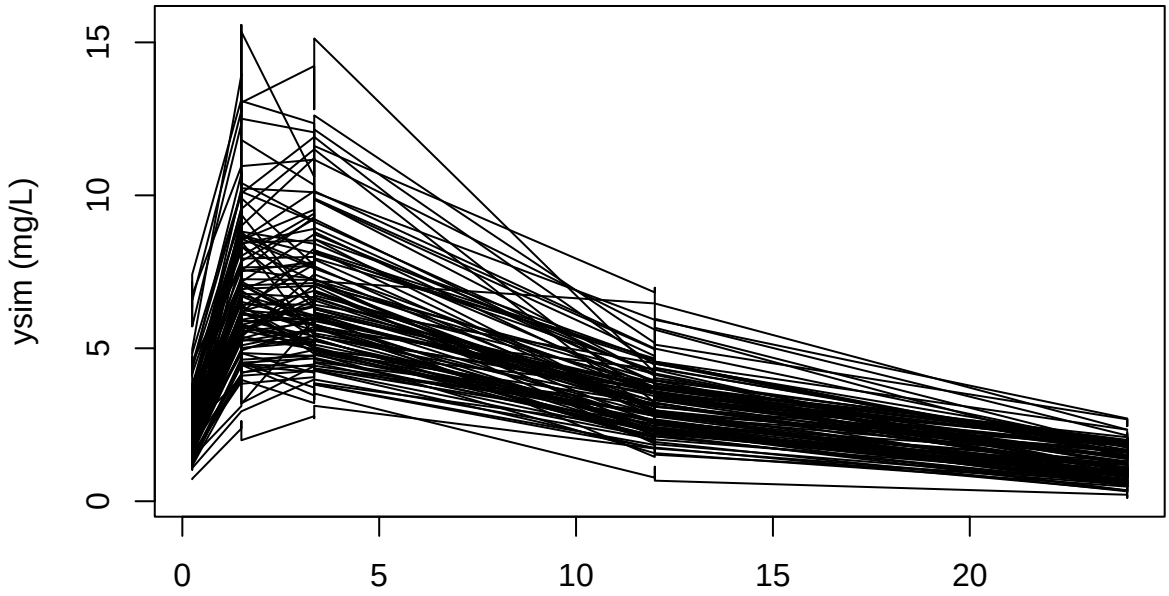
Changes in saemix from previous to new version of the package

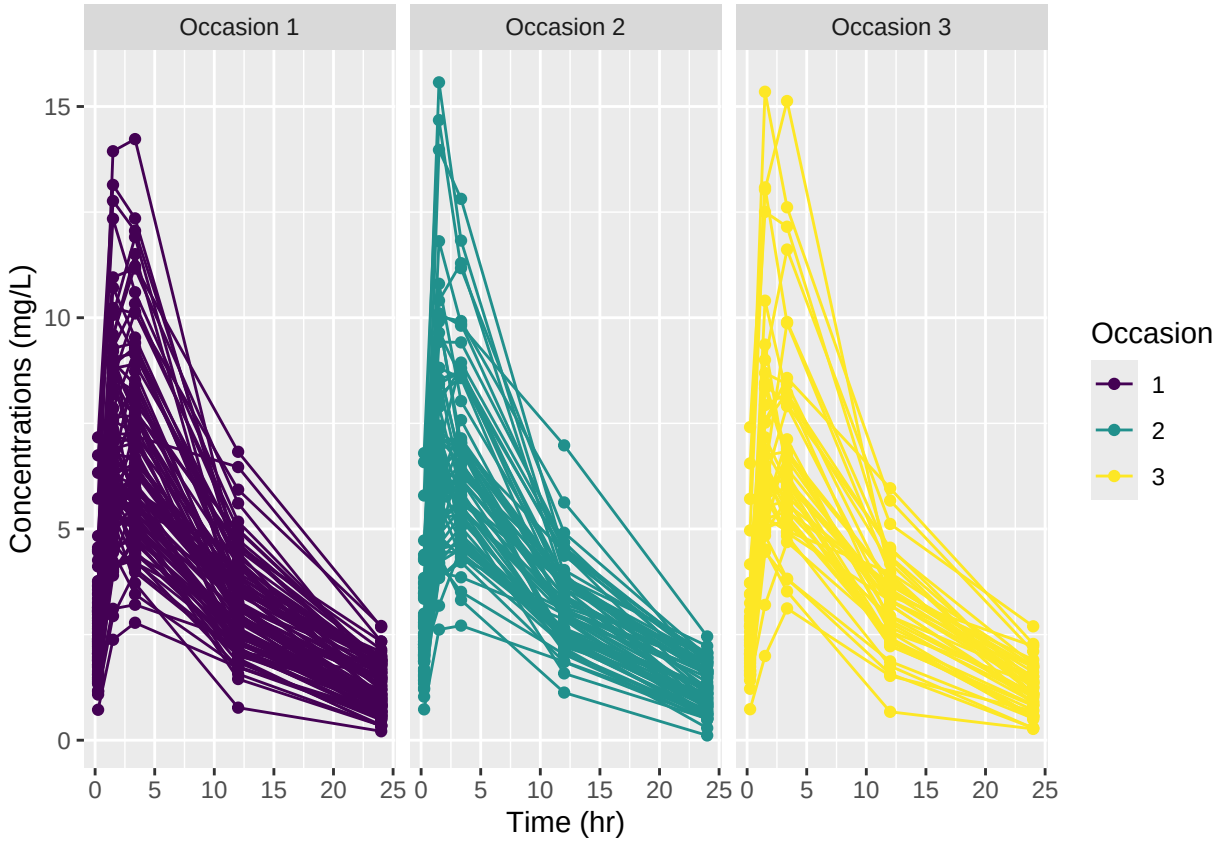
Data object

Functions from the current saemix package

- expected behaviour
  - reading the dataframe should work
  - appropriate columns should be recognised, including occ and ytype
  - plot: seems correct
  - no variability levels
- ToDo:
  - maybe, if: we continue with current version [currently not the plan]
    * add automatic recognition for occ and ytype (currently not read unless explicitely set by user)
    * add "|occ" in the formula

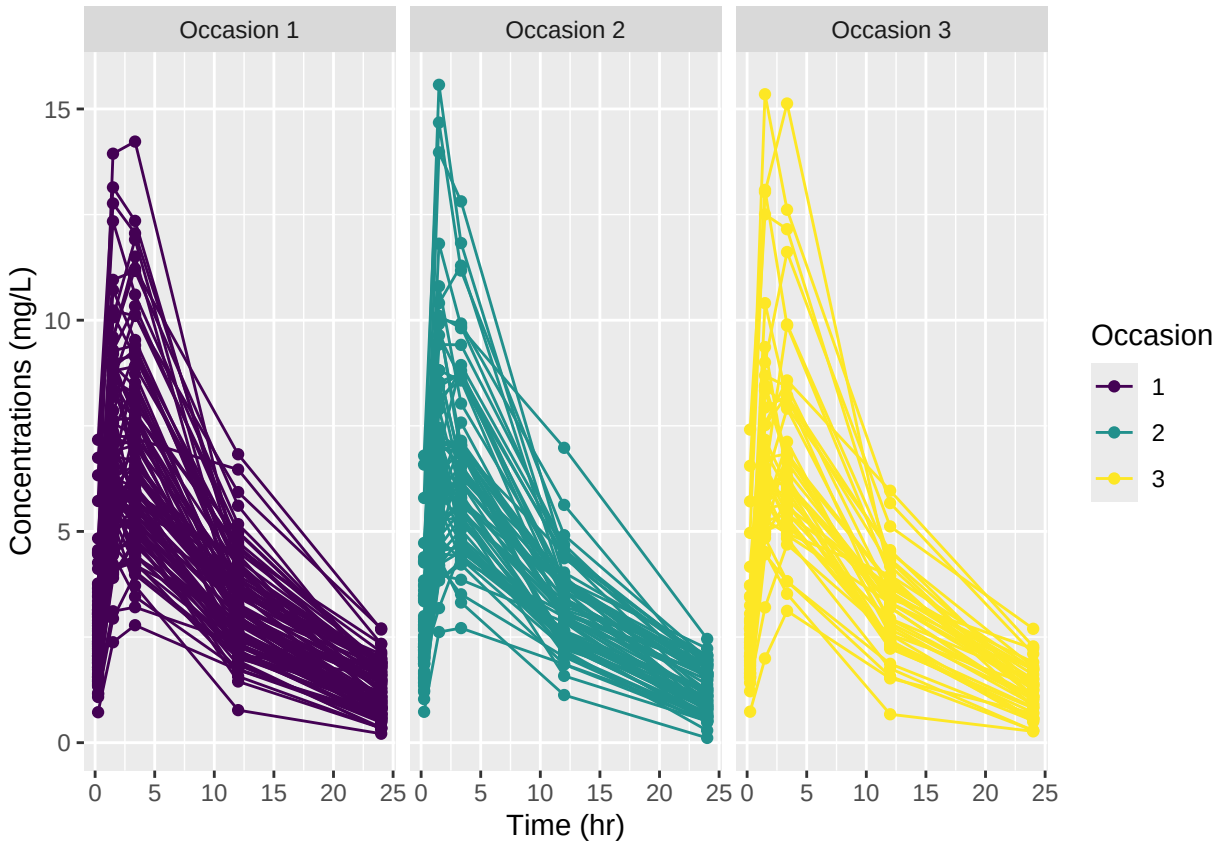
* check nesting order (check occasion is within id)
* dispatch individual plots

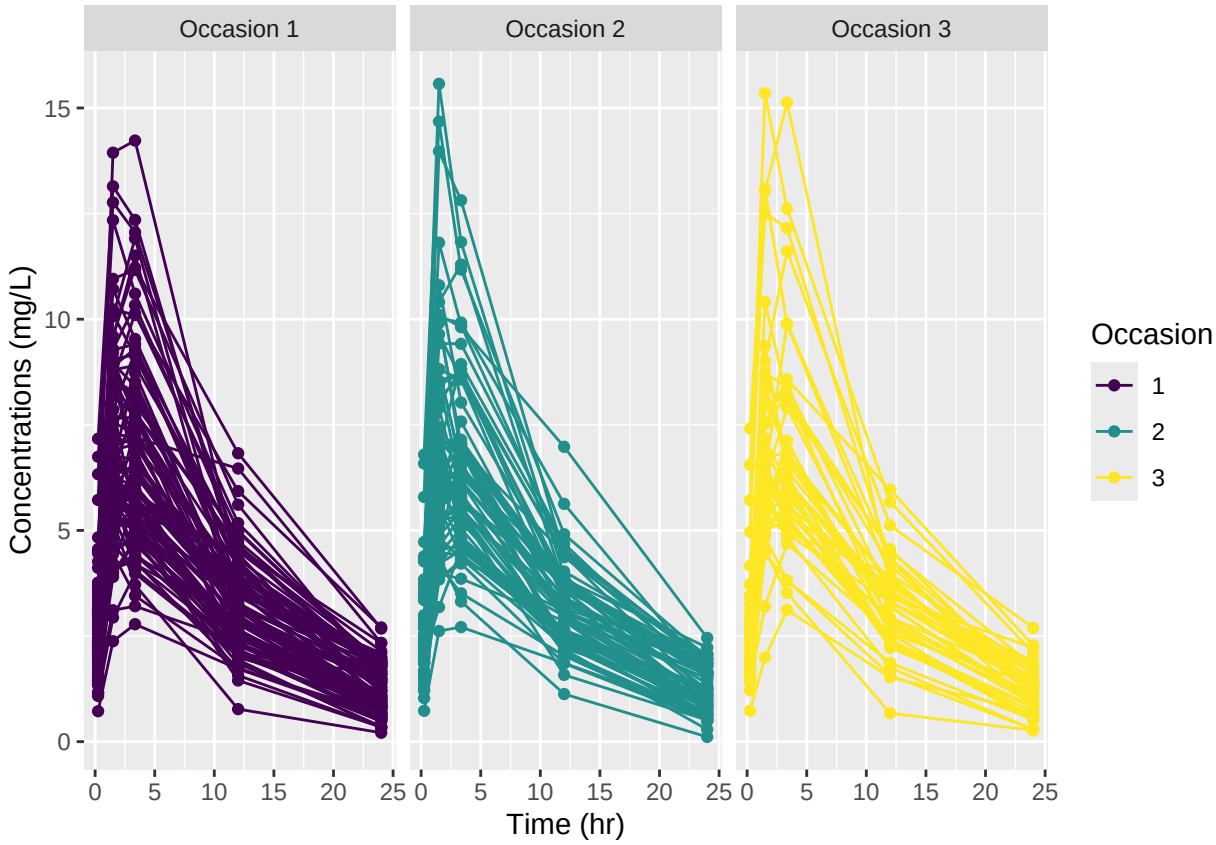




Functions from the new saemix package

- **legacy use**
  - *method*: using the function **saemixData()** as previously
    - objective: seamless transition in legacy mode
  - expected behaviour
    - no error message
    - reading the dataframe should work
    - appropriate columns should be recognised, including occ and ytype
  - **new**: slot outcome
    - outcome are automatically mapped
    - here we expect the first two outcome to be recognised as continuous, and the third as binary will be mapped to categorical
  - systematic check on unchanged slots in the object: **passed**
- **new definition**
  - *method*: same function but explicit definition of outcome (name and type)
  - expected behaviour
    - no error message
    - reading the dataframe should work
    - appropriate columns should be recognised, including occ and ytype
  - **new slot** outcome
    - now we should have named responses with their prespecified type
  - **new slot** varlevel
    - automatically filled in with iiv and iov from name.group and name.occ
    - alternate definition through varlevel
  - systematic check on unchanged slots in the object: **passed**
- ToDo:
  - units for each response
    - add automatic recognition for occ and ytype (currently not read unless explicitely set by user)
    - add "|occ" in the formula
    - check nesting order (check occasion is within id)
    - dispatch individual plots
  - note: see tests for the updated class in corresponding testthat files


Occasion 1
Occasion 2
Occasion 3
Concentrations (mg/L)
Time (hr)
Occasion
1
2
3


Occasion 1
Occasion 2
Occasion 3
Concentrations (mg/L)
Time (hr)
Occasion
1
2
3

Detailed changes to Data object

Model object

- model function used to estimate parameters
  - same arguments psi, id, xidep
  - ytype is a recognised column of xidep so can be used inside the code
- ToDo
  - add occ to xidep if not done yet
  - ToDo: add an option to use a column twice (eg occ could be used both as an indicator of variability level and as a predictor if there is a systematic change we want in the model) ? (another option=simply duplicate the column in the dataset with another name, which is the Monolix standard)
- TBD
  - here occ is also used (but need to make sure it is passed on in the code)
  - do we need occ in the model or should it also be passed 'within' psi ? (need to think; should be passed within psi, but could be passed by the user as a predictor as well)

Functions from the current saemix package

- expected behaviour
  - only IIV taken into account
  - distributions, correlations between parameters handled
  - multiple responses can already be modelled through ytype
- limits for the current package
  - only IIV, no IOV
    - also of course no covariate model for IOV parameters
  - error model: only one taken into account
- ToDo:
  - vector of residual error models
  - maybe, if: we continue with current version [currently not the plan]
    - dispatch individual plots

Functions from the new saemix package

- methods for model object now in a separate file
- **legacy use**
  - *method*: using the **saemixModel()** function as previously
    - objective: seamless transition in legacy mode for models previously handled
  - expected behaviour
    - no error message
  - **new**:
- **new definition without IOV**
  - *method*: same function but explicit definition of parameter distributions
  - expected behaviour
    - no error message
- **new definition with IOV**
  - *method*: include variability levels in the parameter definitions
- ToDo:
  - note: see tests for the updated class in corresponding testthat files
  - create legacy function to use the same instruction as before

```
## dans la méthode pour 'print' avec la signature '"SaemixContinuousOutcome"' : aucune définition de la
```

Detailed changes to Model object - ToDo

- **ToDo** code a legacy model function saemixModel.legacy() for users who wish to use their old code

Additional features (at some point)

- NA
- LOQ